



Relatório de Auditoria de Segurança

Pulso — Gestão de Finanças

Data: 01 de setembro de 2026

Versão auditada: v3.1.9 (commit 2da781d)

Escopo: Frontend SPA, Firebase RTDB, deploy Cloudflare Pages, build Android (Capacitor)

Metodologia: Revisão manual de código-fonte + análise estática

Arquivos verificados: script.js (4685 linhas), login.html, index.html,
database-rules.json, _headers, android/app/build.gradle, service-worker.js

1. Resumo Executivo



Achados por Severidade



- Alta (1)
- Média (3)
- Baixa (1)
- Informativa (1)

Achados por Categoria



Metodologia: Auditoria manual de código-fonte + análise estática. Cada categoria de segurança foi mapeada para a stack: Firebase RTDB rules (isolamento), frontend-only SPA (sem backend), DOMPurify + escapeHTML (XSS). Cada achado foi verificado linha por linha no código real. Não há especulações.

2. Pontos Fortes

- ✓ **Firebase RTDB rules isolam dados por uid** — `auth.uid === $uid` impede acesso cruzado. (`database-rules.json:7-8`)
- ✓ **Todas interpolações em innerHTML usam escapeHTML()** — ~80+ pontos de renderização revisados em `script.js` . Zero XSS refletido/Stored.
- ✓ **DOMPurify + escapeHTML() como defesa em profundidade** — `setHTML()` , `safe()` , `escapeHTML()` aplicados consistentemente. (`script.js:335-353`)
- ✓ **Rate limiting no login** — 5 tentativas, logout 60s, cooldown 2s. (`script.js:720-740`)
- ✓ **Session timeout 15min** — auto-logout por inatividade. (`script.js:742-758`)
- ✓ **CSP headers completos** — X-Frame-Options DENY, HSTS, X-Content-Type-Options, Referrer-Policy, Permissions-Policy. (`_headers:1-8`)
- ✓ **SRI em dependências CDN** — Firebase SDK, DOMPurify, xlsx, chart.js todos com integrity hash. (`login.html:13-16`)
- ✓ **Sanitização de dados de entrada** — `sanitize()` limita campos (user 60 chars, desc 200, val >= 0). (`script.js:444-459`)
- ✓ **Service Worker seguro** — não intercepta cross-origin, não cacheia Firebase. (`service-worker.js:36-43`)
- ✓ **Autenticação dupla** — Email/senha + Google OAuth com tratamento de erros granular. (`script.js:761-822`)

3. Achados Detalhados

Severidade	Arquivo:linha	Descrição
ALTA	android/app/build.gradle:22-24	Keystore password em plaintext no código-fonte storePassword "pulso123" , keyAlias "pulso" , keyPassword "pulso123" — qualquer pessoa com acesso ao repo pode assinar APKs como o autor.
MÉDIA	database-rules.json:16	Regra de leitura aberta em /feedbacks ".read": "auth != null" — qualquer autenticado lê TODOS os feedbacks (uid, email, displayName, photoURL, userAgent de outros).
MÉDIA	script.js:3726-3734	Dados pessoais no payload de feedbacks Payload inclui uid, displayName, email, photoURL, version, userAgent — expostos via regra aberta de leitura.
MÉDIA	login.html:9 , index.html:9 , _headers:8	CSP com unsafe-eval desnecessário 'unsafe-eval' no script-src, mas código não usa eval/new Function.
BAIXA	script.js:7-15	Firebase config hardcoded API key pública por design do Firebase — Application Restrictions devem estar ativas no GCP Console.
INFORMATIVA	script.js:346,352	DOMPurify sem config restritiva DOMPurify.sanitize() usa config padrão. Poderia ter ALLOWED_TAGS customizados.

4. Recomendações Priorizadas

P1 Corrigir AGORA

- Keystore:** Mover senhas para local.properties (já no .gitignore). Se repo foi público, regenerar keystore.

P2 Corrigir em breve

- Feedbacks:** Trocar leitura para "auth.uid === \$data.child('uid').val()" ou remover campos sensíveis do payload.
- CSP:** Remover 'unsafe-eval' de todas as páginas e _headers.
- Payload:** Remover email, photoURL, userAgent do payload de feedbacks.

P3 Melhorias

- Firebase restrictions:** Verificar HTTP referrer restrictions e App Check no GCP Console.
- DOMPurify:** Configurar ALLOWED_TAGS/ALLOWED_ATTR customizados.
- Unsafe-inline:** Migrar scripts inline para arquivos externos + nonce no CSP.

5. Categorias N/A

Permissão no navegador vs backend: Não se aplica. Frontend-only SPA sem backend. Firebase RTDB rules são o equivalente server-side e estão configuradas corretamente.

IDOR: Não se aplica. Operações usam paths com currentUser.uid e regras validam auth.uid === \$uid . Sem endpoints parameterizados por ID de objeto.

6. Issues para o GitHub

[Segurança] Keystore password em plaintext no build.gradle

ALTA labels: security, high

Problema: Senhas do keystore de release hardcoded em `android/app/build.gradle`.

Evidência: `android/app/build.gradle:22-24`

```
signingConfigs {
    release {
        storeFile file("pulso-release.keystore")
        storePassword "pulso123"
        keyAlias "pulso"
        keyPassword "pulso123"
    }
}
```

Impacto: Qualquer pessoa com acesso ao repo pode assinar APKs falsificados. Se repo foi público, keystore comprometido.

Correção: Mover para `local.properties`: `storePassword = project.findProperty('STORE_PASSWORD') ?: ''`

Critérios de aceite:

- ☐ Senhas removidas de build.gradle
- ☐ Senhas em local.properties ou variáveis de ambiente
- ☐ Build de release funciona com novas variáveis
- ☐ Se repo foi público, keystore regenerado

[Segurança] Regra de leitura aberta em feedbacks expõe dados pessoais

MÉDIA labels: security, medium

Problema: Qualquer autenticado lê TODOS os feedbacks (uid, email, displayName, photoURL, userAgent).

Evidência: `database-rules.json:16`, `script.js:3726-3734`

```
"feedbacks": {
  ".read": "auth != null", // qualquer autenticado le tudo
  ".write": "auth != null",
  "$pushId": {
    ".validate": "newData.hasChildren(['uid','message','createdAt'])..."
  }
}

// Payload (script.js:3726-3734)
const payload = {
  uid: currentUser.uid,
  displayName: currentUser.displayName || '',
  email: currentUser.email || '',
  photoURL: currentUser.photoURL || '',
  message: msg,
  userAgent: navigator.userAgent.slice(0,300)
};
```

Correção: `".read": "auth.uid === $data.child('uid').val()"` ou remover campos sensíveis.

- ☐ Regra de leitura restrita ao autor
- ☐ Ou campos sensíveis removidos do payload
- ☐ Teste: usuário A não lê feedbacks de usuário B

[Segurança] CSP inclui unsafe-eval que não é utilizado

MÉDIA labels: security, medium

Problema: `'unsafe-eval'` no script-src sem uso no código (zero chamadas a `eval/new Function`).

Evidência: `login.html:9`, `index.html:9`, `_headers:8`

```
script-src 'self' 'unsafe-inline' 'unsafe-eval' https://www.gstatic.com ...;
```

Correção: Remover `'unsafe-eval'` de todas as diretivas CSP.

- ☐ unsafe-eval removido de login.html, index.html e _headers
- ☐ App funciona sem unsafe-eval
- ☐ Grep confirma zero eval/new Function no código

[Segurança] Firebase API key sem restrições verificáveis

BAIXA labels: security, low

Problema: Firebase config hardcoded — Application Restrictions não verificáveis por código.

Evidência: script.js:7-15

Correção: Configurar HTTP referrer restrictions, App Check, SHA-1 no GCP Console.

- ☐ HTTP referrer restrictions configuradas
- ☐ App Check habilitado
- ☐ SHA-1 do keystore registrado

[Segurança] DOMPurify config padrão + CSP unsafe-inline

INFORMATIVA labels: security, info

Problema: DOMPurify sem ALLOWED_TAGS customizados. CSP com unsafe-inline para scripts.

Evidência: script.js:346 , login.html:9

Correção: Configurar DOMPurify com tags explícitas. Migrar scripts inline para externos + nonce.

- ☐ DOMPurify com ALLOWED_TAGS/ALLOWED_ATTR
- ☐ Scripts inline migrados (opcional)